INTERVIEW GUIDE

Angular

Questions and answers for about 2 years of experience

Standalone components · Signals · Forms · Routing · HTTP

Prepared 31 October 2026. Examples target Angular 22. New apps are standalone, zoneless, and OnPush by default. Many codebases a two-year developer has shipped are still on NgModules, Zone.js, and Reactive Forms. Know both, and say which one the team is on.

How to use these answers

A two-year interview is scoring whether you have shipped screens, not whether you can redesign the framework. Answer in about a minute: what it is, how you use it in a feature, and one bug you would catch in review. Then stop. If they want the old API and you only gave the new one, add one sentence about the code you would still find in an existing app.

1. First sentence: the decision or the definition.
2. Second beat: a small orders screen, a form, or an HTTP call.
3. Third beat: the mistake that leaks a subscription, trusts the client, or refreshes too often.
4. Stop. Let them ask for signals, NgRx, or SSR.

What this level includes

You can build a feature against an existing API: a routed page, a form, a service, loading and error state, and a guard. You can explain change detection well enough to use OnPush and signals without folklore. You are not expected to write a custom render strategy, design a micro-frontend platform, or own the design system.

The running example

Most answers use an orders page. The user opens /orders, the page loads a page of orders from a .NET API, and a form creates an order. The API returns 401 when the token is missing and 400 with field errors when the body is invalid. The Angular app does not decide the price.

Framework and components

What is Angular, and how is it different from AngularJS?

Angular is a TypeScript framework for single-page applications. A component owns a template, styles, and a class. The framework routes URLs to components, injects services, and updates the DOM when state

changes. AngularJS, the 1.x framework, is a different product: controllers, $scope, and digest cycles. They do not share a runtime. If a resume says Angular and the job is AngularJS, say so early.

```
@Component({
  selector: 'app-order-page',
  template: `<h1>Orders</h1>`,
})
export class OrderPage {}
```

Say this

Angular is the component framework. AngularJS is the older digest-cycle framework. I have shipped Angular. I do not pretend an AngularJS controller is the same thing.

What is a component?

A component is the unit of UI. The class holds state and handlers. The template declares the HTML. The selector is how a parent places it. In a current app the component is standalone, so it lists the other components, directives, and pipes it uses in imports. The template cannot use something the component did not import.

```
@Component({
  selector: 'app-order-page',
  imports: [OrderTable, CurrencyPipe],
  templateUrl: './order-page.html',
  styleUrl: './order-page.css',
})
export class OrderPage {
  private readonly api = inject(OrderApi);
  readonly orders = signal<Order[]>([]);
}
```

What breaks

A component that fetches data, maps DTOs, and formats currency in one class is hard to test. Keep HTTP in a service. Keep the component on state the template needs.

Say this

A component is template plus class, and it imports what the template uses. I keep HTTP in a service so the component stays about the screen.

Standalone components or NgModules?

A standalone component declares its own dependencies. bootstrapApplication starts the app with providers, with no root NgModule. An NgModule groups declarations, imports, and providers. That is what you still see in apps started before standalone was the default. New Angular apps do not need AppModule. I would not add a new NgModule to a standalone app unless a library still forces one.

```
bootstrapApplication(App, {
  providers: [
    provideRouter(routes),
    provideHttpClient(withInterceptors([authInterceptor])),
  ],
});

// Older app, still common in a job you join:
@NgModule({
  declarations: [OrderPage],
  imports: [CommonModule, HttpClientModule, RouterModule],
  bootstrap: [App],
})
export class AppModule {}
```

Say this

New code is standalone and bootstraps with bootstrapApplication. I can still read an NgModule. I do not mix a new feature module into a standalone app without a reason.

What are the kinds of binding?

Interpolation prints a value into text. Property binding sets a DOM or component property. Event binding listens. Two-way binding is property plus event, which is what a signal model or ngModel is doing. Attribute binding sets an HTML attribute when the DOM property is the wrong target, such as aria labels. Class and style bindings are the ones I use instead of concatenating class strings in the component.

```
<p>Total {{ order().total }}</p>
<button [disabled]="saving()" (click)="save()">Save</button>
<input [value]="query()" (input)="query.set($any($event.target).value)" />
<div [class.is-late]="order().status === 'Late'"></div>
```

Say this

Interpolation shows a value, brackets set a property, parentheses listen. I do not put a function with a heavy loop in interpolation, because the template reads it whenever that view is checked.

How does a child receive data and tell the parent?

The current API is input, output, and model. input is data from the parent. input.required fails the build if the parent forgets it. output is an event the child emits. model is a two-way input: the parent can use a banana-in-a-box binding and the child can write back. The older @Input and @Output decorators still compile and still fill existing code. I use the function APIs in new components.

```
export class OrderRow {
  readonly order = input.required<Order>();
  readonly selected = output<number>();

  choose() {
```

```
    this.selected.emit(this.order().id);
  }
}

// parent template
<app-order-row [order]="order" (selected)="open($event)" />

// two-way quantity
export class QtyField {
  readonly quantity = model(1);
}
```

What breaks

Mutating an input object in the child surprises the parent, because both sides hold the same object. Emit an event or write through model. Do not reach into the parent's service from a presentational row if a simple output will do.

Say this

input down, output up. model when the parent and child edit the same piece of state. I do not mutate the parent's object inside the child.

Which lifecycle hooks do you actually use?

ngOnInit runs once after the first inputs are set. I use it when setup must wait for those inputs. Constructor work is for inject and for creating signals, not for HTTP, because the template is not ready and inputs may still change. ngOnChanges runs when an input reference changes. With signal inputs I usually use a computed or an effect instead of ngOnChanges. ngOnDestroy is cleanup: unsubscribe, remove a listener. ngAfterViewInit runs after the view children exist, which is when a ViewChild is safe to read. I do not implement hooks I do not need.

```
export class OrderDetail {
  readonly id = input.required<number>();
  private readonly api = inject(OrderApi);

  constructor() {
    effect(() => {
      const id = this.id();
      untracked(() => this.api.load(id));
    });
  }
}
```

The effect reads the id signal. When the route changes the id, the effect runs again. untracked keeps the load from becoming a dependency if the service writes other signals. For a one-shot call I still see ngOnInit in older components, and that is fine.

Say this

I set up dependencies in the constructor, I start work from inputs in ngOnInit or an effect, and I release subscriptions in ngOnDestroy or with takeUntilDestroyed. I do not fetch in the constructor.

@if, @for, and @switch

Built-in control flow is the syntax I write now. @if shows a branch. @for repeats, and track is required so Angular can match rows when the list changes. @empty covers an empty list. @switch picks one case. *ngIf, *ngFor, and *ngSwitch still work if CommonModule is imported. I can read them. I do not add them to a new template.

```
@if (loading()) {
  <p>Loading orders...</p>
} @else if (error()) {
  <p>{{ error() }}</p>
} @else {
  @for (order of orders(); track order.id) {
    <app-order-row [order]="order" />
  } @empty {
    <p>No orders yet.</p>
  }
}

@switch (order().status) {
  @case ('Pending')
  @case ('Processing') {
    <span>In progress</span>
  }
  @case ('Shipped') {
    <span>On the way</span>
  }
  @default {
    <span>Unknown</span>
  }
}
```

What breaks

track order, or track $index on a list that is reordered, makes Angular reuse the wrong DOM row. Track a stable id. A method call in the @if condition that also writes state will surprise you, because the template reads it during check.

Say this

I use @if and @for with track on a stable id. *ngFor is what I still find in older templates. Tracking the object identity or the index is how a list flickers or shows the wrong row.

What is a pipe, and when is it impure?

A pipe formats a value in the template. currency, date, async, and json are the built-ins I use. A pure pipe runs when the input reference changes. An impure pipe runs on every change detection, which is expensive. async is impure on purpose: it subscribes and marks the view when the observable or promise emits. I would rather format in a computed signal if the rule is business logic, and keep the pipe for display.

```
<p>{{ order().total | currency:'USD' }}</p>
<p>{{ createdAt() | date:'medium' }}</p>

// Prefer this to an impure pipe that filters a large list:
readonly openOrders = computed(() => this.orders().filter(o => o.status === 'Open'));
```

Say this

A pipe formats. Pure pipes skip work when the input is the same reference. I do not filter a big array with an impure pipe. I filter in a computed signal.

ng-content, ng-template, and ng-container

ng-content projects the parent's inner HTML into the child. That is how a card wraps whatever the page puts inside it. ng-template is a template that does not render until someone stamps it, which is what structural directives and explicit TemplateRef use. ng-container groups nodes without adding a DOM element, so a layout does not gain an extra div. @if already groups without a container, so I use ng-container less than I used to.

```
@Component({
  selector: 'app-card',
  template: `<section class="card"><ng-content /></section>`,
})
export class Card {}

<app-card><app-order-row [order]="order" /></app-card>
```

Say this

ng-content is the slot. ng-template is inert until it is rendered. ng-container groups without a host element.

What is viewChild for?

viewChild queries one element or component in this component's template. It is a signal in current Angular, so I read it by calling it, and it is undefined until the view exists. I use it for a focus call or a child component method. I do not use it to pull data out of a child that should have been an output. contentChild is the same idea for projected content.

```
readonly nameInput = viewChild<ElementRef<HTMLInputElement>>('nameInput');
```

```
focusName() {
  this.nameInput()?.nativeElement.focus();
}

// template: <input #nameInput />
```

Say this

viewChild is a handle to my own template. It is empty before the view is created. If I am using it to read a child's state, I wanted an output instead.

Dependency injection and services

How does dependency injection work?

A component or service asks for a type. Angular walks the injector tree and returns the instance it finds. The root injector lives for the app. A route can have its own injector. A component can too, with providers on the component, and that instance is created per component and destroyed with it. inject reads from the current injection context, which is the constructor, a field initializer, or a factory. I prefer inject in new code because the dependency is next to the field and inheritance stays simpler.

```
export class OrderPage {
  private readonly api = inject(OrderApi);
  private readonly router = inject(Router);
}
```

Say this

I ask for a type, Angular finds the instance. Root lives for the app. A component provider lives with that component. I use inject in new code.

What is @Service, and when do you still use @Injectable?

Angular 22 added @Service for the common case: a singleton available to the whole app. It replaces @Injectable with providedIn root for that simple service. @Injectable remains when you need a factory, useClass, useExisting, or a component-level provider. Existing apps are full of @Injectable. Both are valid. I name the one the codebase already uses, and I use @Service when I add a plain root service in a new app.

```
import { Service } from '@angular/core';

@Service()
export class OrderApi {
  private readonly http = inject(HttpClient);

  list() {
    return this.http.get<Order[]>('/api/orders');
  }
}
```

```
// Still the right tool when the token is configured:
@Injectable()
export class ApiConfig {
  constructor(private readonly baseUrl: string) {}
}
```

What breaks

Providing the same service in root and again on a component creates two instances. State set in one is invisible to the other. That bug looks like a refresh problem.

Say this

@Service is the root singleton. @Injectable stays for factories and for a provider that is not the root. Two providers means two instances.

Where should a service live?

An API client and an auth token belong at the root, one instance, so every page sees the same token. A form wizard that must forget its state when the user leaves can be provided on the route or the component. I do not put HTTP URLs and caching policy inside the component class.

```
@Component({
  selector: 'app-wizard',
  providers: [WizardState],
  template: `...`,
})
export class Wizard {}
// WizardState is created with this Wizard and destroyed with it.
```

Say this

Root for app-wide clients and auth. Component or route for state that must die with the screen. I can point at which injector owns the instance.

What is injectAsync for?

injectAsync loads a service the first time you need it, from a dynamic import. The initial bundle does not include that code. I would use it for a heavy export or a chart library behind a button, not for OrderApi, which the first page needs immediately. The service has to be auto-provided, which @Service does.

```
export class ReportPage {
  private readonly exporter = injectAsync(() => import('./report-exporter'));

  async export() {
    const exporter = await this.exporter();
    exporter.export();
```

```
  }
}
```

Say this

injectAsync code-splits a service until it is used. I use it for rare heavy work. The order API stays a normal inject.

RxJS and HTTP

Why observables, if promises exist?

A promise is one future value. An observable is a stream: zero, one, or many values, and it can be cancelled. HTTP happens to be one response, but valueChanges, router events, and websocket messages are streams. HttpClient returns an observable so interceptors, retry, and switchMap compose. async and await are still what I use inside a loader once I have decided to do one call.

```
this.query$
  .pipe(
    debounceTime(300),
    distinctUntilChanged(),
    switchMap(query => this.api.search(query)),
  )
  .subscribe(rows => this.orders.set(rows));
```

Say this

A promise is one value. An observable is a cancellable stream. I use the stream when the user can type again, navigate away, or emit more than once.

switchMap, mergeMap, concatMap, or exhaustMap?

All four map an outer value to an inner observable. switchMap unsubscribes from the previous inner call. That is search: the response for 'ord' must not overwrite the response for 'order'. mergeMap runs inners together. That is saving several independent files. concatMap waits for each inner to finish. That is a queue of writes that must stay in order. exhaustMap ignores new outers until the current inner finishes. That is a save button, so a double click does not start a second POST.

```
// Search: only the latest query matters.
switchMap(query => this.api.search(query))

// Save button: ignore clicks while the POST is in flight.
exhaustMap(() => this.api.create(this.draft()))
```

What breaks

mergeMap on a search box shows stale rows when a slow response returns last. switchMap on a POST can cancel the request in the browser and still leave the server creating the order. Do not cancel a create.

Say this

switchMap for the latest search. exhaustMap for a button that must not double-submit. concatMap when order matters. mergeMap when the calls are independent.

Subject or BehaviorSubject?

A Subject emits only to subscribers who are already listening. A late subscriber misses the last value. A BehaviorSubject stores the current value and replays it on subscribe, so it needs an initial value. I used BehaviorSubject for a cart or a current user before signals. In new code I keep that state in a signal. I still use a Subject as a trigger, such as 'refresh clicked', when I am staying in RxJS.

```
private readonly userSubject = new BehaviorSubject<User | null>(null);
readonly user$ = this.userSubject.asObservable();

// New code:
readonly user = signal<User | null>(null);
```

Say this

BehaviorSubject has a current value. Subject does not. New shared state is a signal. I do not hand the Subject itself to callers, or they can call next from anywhere.

How do you avoid a leaked subscription?

The async pipe subscribes in the template and unsubscribes when the view is destroyed. That is the safest default for a stream the template displays. If I subscribe in the class, I use takeUntilDestroyed. Calling it in the constructor or a field initializer can see the destruction of this component. Outside that injection context I pass a DestroyRef. Nested subscribes are the review comment: the inner subscription is easy to leak, and switchMap already expressed it.

```
export class OrderPage {
  private readonly api = inject(OrderApi);
  readonly orders = signal<Order[]>([]);

  constructor() {
    this.api.list()
      .pipe(takeUntilDestroyed())
      .subscribe(rows => this.orders.set(rows));
  }
}
```

```
// Template alternative, no manual subscribe:
// {{ orders$ | async }}
```

Say this

async pipe or takeUntilDestroyed. I do not nest subscribe. If the template can own the stream, I let it.

How do you call the API and show errors?

HttpClient is provided with provideHttpClient. A service method returns the observable or, in new code, I convert one shot into a signal the page can read. The page needs three states: loading, the rows, and an error string a human can read. A 400 from the API is a client problem. A 401 is an auth problem. A 500 or a status 0 is ours or the network. I do not show exception text from the server in the template.

```
list(): Observable<Order[]> {
  return this.http.get<Order[]>('/api/orders');
}

load() {
  this.loading.set(true);
  this.error.set(null);
  this.api.list().pipe(takeUntilDestroyed(this.destroyRef)).subscribe({
    next: rows => this.orders.set(rows),
    error: (err: HttpErrorResponse) => {
      this.error.set(err.status === 0 ? 'Cannot reach the server.' : 'Could not load orders.');
    },
    complete: () => this.loading.set(false),
  });
}
```

httpResource is the newer declarative read. The URL can depend on a signal, and the resource exposes loading, value, and error. I use it when the screen is a query that should rerun as the signal changes. I still use HttpClient for a POST, because creating an order is an action, not a value the template is watching.

```
selectedId = signal(10);
order = httpResource<Order>(() => `/api/orders/${this.selectedId()}`);
```

Say this

The service performs HTTP. The page shows loading, data, and a safe error. httpResource fits a read that follows a signal. HttpClient fits the command. I do not print the raw server error.

What does an interceptor do?

An interceptor sees every HttpClient request and response. I use one to attach the bearer token and one to turn a 401 into a redirect to login. I do not put business validation there. Functional interceptors are functions registered with withInterceptors. The old class interceptor still appears in NgModule apps.

```
export const authInterceptor: HttpInterceptorFn = (req, next) => {
  const token = inject(AuthStore).token();
  const authed = token
    ? req.clone({ setHeaders: { Authorization: `Bearer ${token}` } })
    : req;
  return next(authed);
};

provideHttpClient(withInterceptors([authInterceptor]))
```

What breaks

Mutating req.headers in place does nothing useful. clone the request. Attaching the token to a login request can also be wrong. Skip the URL that issues the token. An interceptor that navigates on every error, including 400, sends the user to login when the form was invalid.

Say this

Interceptors are for cross-cutting HTTP: the token, a correlation id, a 401 redirect. I clone the request. Validation stays in the form and the API.

Forms

Which form API do you use?

Signal Forms are the forms API in current Angular. A writable signal holds the model. form builds a field tree. The template binds an input with formField. Reactive Forms, FormGroup and FormControl, are what most two-year codebases still contain, and I can maintain them. Template-driven forms use ngModel in the template. I liked them for a tiny login and I stopped using them once validation and tests mattered. New forms start as Signal Forms. I do not rewrite a large Reactive Form in the same pull request as a feature.

```
import { form, FormField, required, min } from '@angular/forms/signals';

export class CreateOrder {
  readonly model = signal({ sku: '', quantity: 1 });
  readonly orderForm = form(this.model, schema => {
    required(schema.sku, { message: 'SKU is required' });
    min(schema.quantity, 1, { message: 'Quantity must be at least 1' });
  });

  save() {
    if (this.orderForm().invalid()) {
      return;
    }
    this.api.create(this.model());
  }
}

// template
```

```
<input [formField]="orderForm.sku" />
@if (orderForm.sku().invalid() && orderForm.sku().touched()) {
  @for (error of orderForm.sku().errors(); track error.kind) {
    <span>{{ error.message }}</span>
  }
}
<button [disabled]="orderForm().invalid()">Create</button>
```

The Reactive Form I would still recognize on an existing screen looks like this.

```
readonly form = this.fb.nonNullable.group({
  sku: ['', Validators.required],
  quantity: [1, [Validators.required, Validators.min(1)]],
});

save() {
  if (this.form.invalid) {
    this.form.markAllAsTouched();
    return;
  }
  this.api.create(this.form.getRawValue());
}
```

Say this

New form: a signal model and form from @angular/forms/signals. Existing form: FormGroup. I disable submit when the form is invalid, and I show errors after touch so the page does not shout on first paint.

How do you validate, and where do server errors go?

Client validation is for fast feedback: required, min, email, a match between password fields. The server is the authority. A 400 with field names comes back onto the form so the user sees them next to the control. I do not trust the client price or tenant id. Those are not form fields the browser may edit.

```
// Signal form: read the model only after invalid() is false.
create() {
  const body = { sku: this.model().sku, quantity: this.model().quantity };
  this.api.create(body).subscribe({
    next: order => this.router.navigate(['/orders', order.id]),
    error: (err: HttpErrorResponse) => {
      if (err.status === 400) {
        this.error.set('Check the SKU and quantity.');
      }
    },
  });
}
```

Say this

The browser checks shape. The API checks the business rule. I map a 400 back to the form. I never send a total the client typed as the price.

Routing

How does routing work?

provideRouter registers routes. A route maps a path to a component. routerLink navigates without a full page load. router-outlet is where the child component renders. Route params are the path pieces, such as /orders/15. Query params are the search string, such as ?status=Open. I read params as signals from the current route input APIs, or from ActivatedRoute in older code. I do not parse window.location in a component.

```
export const routes: Routes = [
  { path: 'orders', component: OrderPage },
  { path: 'orders/:id', component: OrderDetail },
  { path: 'login', component: LoginPage },
  { path: '', pathMatch: 'full', redirectTo: 'orders' },
];

<a routerLink="/orders">Orders</a>
<router-outlet />
```

Say this

Routes map a URL to a component. routerLink changes the URL. The outlet hosts the page. Params are the path. Query params are the filter.

How do you lazy-load a page?

loadComponent imports the page only when the user navigates there. The main bundle stays smaller. I lazy-load feature pages, not the shell and not a button the home page always shows. A failed import is a broken route, so the chunk has to be deployed with the app.

```
{
  path: 'orders',
  loadComponent: () => import('./orders/order-page').then(m => m.OrderPage),
}
```

Say this

loadComponent splits that page into its own chunk. I lazy-load features the user may never open. I keep the first screen eager.

What is a guard?

A guard decides if a navigation may continue. canActivate runs before the route activates. I use it so an anonymous user cannot open /orders. It returns true, a UrlTree, or a redirect. It is not a security boundary. The API must still reject a missing token. A guard that only hides a button is UI. Anyone can call the API.

```
export const authGuard: CanActivateFn = () => {
  const auth = inject(AuthStore);
  const router = inject(Router);
  return auth.isLoggedIn() ? true : router.createUrlTree(['/login']);
};

{ path: 'orders', canActivate: [authGuard], loadComponent: () => import('./orders/order-
page').then(m => m.OrderPage) }
```

canDeactivate asks before leaving a dirty form. A resolver preloads data before the component. I often skip resolvers and load in the component so the shell can show a spinner. A resolver that hangs blocks the whole navigation.

Say this

The guard protects the route. The API protects the data. canDeactivate protects a dirty form. I do not treat a hidden button as authorization.

Change detection and signals

How does change detection work?

Angular updates bindings, then the DOM, when it knows state may have changed. With Zone.js, patched async work schedules a sweep of the component tree. Eager strategy, the old name was Default, checks the component whenever that sweep runs. OnPush checks a component when an input reference changes, when an event fires in its template, when an async pipe emits, when a signal it read changes, or when you call markForCheck. Angular 22 new apps are OnPush by default, and zoneless is the default from Angular 21, so Zone.js is not what schedules the sweep. You notify Angular by updating a signal, using the async pipe, or marking the view.

```
// New app: OnPush is already the default. This is explicit only on an old component.
@Component({
  selector: 'app-order-total',
  changeDetection: ChangeDetectionStrategy.OnPush,
  template: `<p>{{ total() }}</p>`,
})
export class OrderTotal {
  readonly lines = input.required<Line[]>();
  readonly total = computed(() => this.lines().reduce((sum, line) => sum + line.price, 0));
}
```

What breaks

Mutating an array in place, orders.push(row), does not change the reference. OnPush and a signal that still holds the same array will not show the row. Set a new array, or update the signal. Calling detectChanges in a loop hides the real bug.

Say this

OnPush updates when inputs, template events, async pipes, or signals say so. New Angular is OnPush and zoneless by default. The old default is now called Eager, and it checked more often because of Zone.js. I replace the array instead of mutating it.

What is a signal?

A signal holds a value and notifies readers when it changes. You read it by calling it. set replaces the value. update computes the next value from the current one. computed derives a value and caches it until a dependency changes. The template is a reactive context, so reading a signal in the template tracks it. effect is for a side effect when signals change. I keep effects rare, because an effect that writes another signal is easy to loop.

```
readonly quantity = signal(1);
readonly price = signal(20);
readonly total = computed(() => this.quantity() * this.price());

addOne() {
  this.quantity.update(q => q + 1);
}
```

Say this

I read a signal by calling it. computed is derived state. set and update write. An effect is a side effect, not where I calculate the total.

When do you still use RxJS instead of signals?

Signals are the state the template reads. RxJS is still how I compose events over time: debounce a search box, switchMap to HTTP, retry, race a timeout. I convert the result into a signal at the edge with toSignal, or I use httpResource when the read is the whole story. I do not wrap every HTTP call in both a Subject and a signal.

```
private readonly query = new Subject<string>();
readonly results = toSignal(
  this.query.pipe(
    debounceTime(300),
    distinctUntilChanged(),
    switchMap(text => this.api.search(text)),
  ),
  { initialValue: [] as Order[] },
);

search(text: string) {
  this.query.next(text);
}
```

Say this

Signals hold the screen state. RxJS composes the stream. I switch to a signal where the template reads, and I do not keep two copies of the same list.

How do a parent and a child share a larger state?

One order row uses input and output. A cart or the signed-in user is a root service with signals, because many pages read it. I do not introduce NgRx for a single page. NgRx, or another store, earns its place when many features dispatch changes to the same state and the team wants an explicit event for each change. At two years I would rather show a small signal service I understand than a store I copied.

```
@Service()
export class CartStore {
  readonly lines = signal<Line[]>([]);
  readonly count = computed(() => this.lines().length);

  add(line: Line) {
    this.lines.update(lines => [...lines, line]);
  }
}
```

Say this

Input and output for a row. A signal service for state several pages share. NgRx when the team already uses it or the event history is the requirement. I do not start a new app on a store I cannot draw.

A feature, end to end

How would you protect an orders page with a token?

The login form posts credentials. The API returns a token. AuthStore keeps it in a signal and in memory. I do not put a long-lived token in localStorage unless the product accepts the XSS risk, and I say that tradeoff. The interceptor adds the bearer header. The guard redirects if the signal is empty. The API still returns 401 if the token is forged. On 401 I clear the store and go to login.

```
@Service()
export class AuthStore {
  readonly token = signal<string | null>(null);
  readonly isLoggedIn = computed(() => this.token() !== null);

  setToken(token: string) {
    this.token.set(token);
  }

  clear() {
    this.token.set(null);
```

```
  }
}
```

Say this

The guard is the UX. The interceptor attaches the token. The API is the security. A 401 clears the session. Hiding the menu is not auth.

How do you keep a list fast?

Track by id so DOM nodes are reused. Do not filter a huge array in the template with a method. Page on the server when the list can grow. OnPush, which new apps already use, skips checks when that subtree had no signal or input change. A pure pipe or a computed total avoids recalculating on unrelated keystrokes. I measure before I virtualize. CDK virtual scroll is the tool when the user really renders thousands of rows, not when the API should have paged.

```
@for (order of orders(); track order.id) {
  <app-order-row [order]="order" />
}
```

Say this

Stable track, server paging, and computed values. Virtual scroll is for a list we truly render. I do not virtualize a bug that should have been a page size.

How do you stop XSS?

Angular escapes interpolation, so a SKU that contains a script tag is text, not HTML. innerHTML and bypassSecurityTrustHtml are the holes. I do not bind a product description as HTML unless it was sanitized on the server and I still treat it as hostile. URLs from the API go through a check before they become href or src. The token does not get written into the DOM.

```
<p>{{ order.name }}</p>
<!-- Review reject: [innerHTML]="order.description" -->
```

Say this

Interpolation is escaped. innerHTML and bypassSecurityTrust are the review flags. I do not render API HTML because the design asked for bold.

How do you test a service and a component?

A service test calls the method and asserts the HTTP request with HttpTestingController, or asserts a pure mapping with no TestBed at all. A component test uses TestBed, sets inputs, clicks, and asserts the DOM. I test the behavior the user cares about: invalid form does not call create, a 400 shows the message. I do not assert a private signal's existence as the goal.

```
it('does not create when the sku is empty', () => {
  const api = jasmine.createSpyObj('OrderApi', ['create']);
  TestBed.configureTestingModule({
    imports: [CreateOrder],
    providers: [{ provide: OrderApi, useValue: api }],
  });
  const fixture = TestBed.createComponent(CreateOrder);
  fixture.detectChanges();
  fixture.componentInstance.save();
  expect(api.create).not.toHaveBeenCalled();
});
```

Say this

I test the user outcome: bad form does not post, a failed load shows an error. HttpTestingController asserts the request. I mock the service at the boundary, not HttpClient and the router and the store all at once, unless the test is about those.

What do you look for in an Angular pull request?

- The page has a loading state and an error state, not only the happy list.
- Subscriptions die with the component, or the template uses the async pipe.
- A search uses switchMap. A create uses exhaustMap or a disabled button.
- @for tracks a stable id.
- The client does not send price, role, or tenant id as authority.
- A new component imports what it uses and does not reach into another feature's private files.
- A 401 path and a guard agree. The API is still the real check.

Say this

I review state, cancellation, the list track, and whether the client is trusted with a field the server should own.

Questions they use as follow-ups

Angular or React?

I answer from the job, not from a team sport. Angular gives me routing, forms, HTTP, and DI as one framework, which is why a large line-of-business app often picks it. React gives me a view library and a choice of everything else. I have more Angular screen time, so I am faster there. I can learn the other. I do not insult the one their codebase uses.

Say this

Angular is the full framework I have shipped. I choose it when the team wants one opinionated stack. I do not turn the question into a fight.

What is the difference between ngOnChanges and a signal input?

ngOnChanges hands you a SimpleChanges map when an @Input reference changes. A signal input is already a signal. computed and effect read it. I do not implement ngOnChanges to copy an input into another field. That second field goes stale.

Say this

Signal inputs are values I can compute from. ngOnChanges is the older notification. Copying an input into a local field is how the screen shows yesterday's order.

What is a directive?

A directive is a behavior on an element without its own template. Attribute directives change appearance or listen, such as a highlight or routerLink. Structural directives change the structure. *ngIf was one. Components are directives with a template. I write a directive when two components need the same element behavior and a component wrapper would add a useless tag.

```
@Directive({ selector: '[appAutofocus]' })
export class Autofocus {
  private readonly el = inject(ElementRef<HTMLElement>);
  constructor() {
    afterNextRender(() => this.el.nativeElement.focus());
  }
}
```

Say this

A directive adds behavior to an element. A component is a directive with a template. I extract a directive when the behavior is about the element, not about a new layout.

What belongs in an environment file?

The API base URL and a feature flag that differs by deploy. Not secrets. An Angular app is downloaded to the browser, so any key in the bundle is public. The token comes from the login response. The production build replaces the environment at build time, or, better on a serious team, the server injects config at startup so one build can ship to more than one environment.

Say this

The base URL can be in config. A secret cannot. The bundle is public. I prefer one build and runtime config when the pipeline can do it.

How do you handle a slow search box?

debounceTime waits until the user pauses. distinctUntilChanged drops the same text. switchMap cancels the previous HTTP call in the browser and ignores its result. A loading flag stays true until the latest request settles. I set a minimum length so one letter does not hit the API.

```
this.query.pipe(
  debounceTime(300),
  map(text => text.trim()),
  distinctUntilChanged(),
  filter(text => text.length >= 2),
  switchMap(text => this.api.search(text)),
  takeUntilDestroyed(),
).subscribe(rows => this.orders.set(rows));
```

Say this

Debounce, ignore duplicates, switchMap so the slow response cannot win, and do not search on one character.

Short answers

Question	Answer
AOT	The template compiler runs at build time. Errors show in the build, and the browser does less work.
Tree shaking	The build drops code no import reaches. Standalone imports make that accurate. A barrel file that imports everything can defeat it.
Eager change detection	The old Default strategy. Zone-driven checks visit the component often. OnPush is the default in a new Angular 22 app.
Zone.js	It patches async APIs and triggered change detection. Zoneless apps do not include it. Signals and events notify Angular instead.
Smart and presentational	A page loads data. A row displays an input and emits an output. The row does not inject HttpClient.
track	The identity of a row in @for. Use the id, not the index, when rows can move.
async pipe	Subscribes in the template and unsubscribes with the view. Also marks OnPush for check.
takeUntilDestroyed	Completes the subscription when the component is destroyed.
providedIn root	One instance for the app. @Service is the Angular 22 spelling of that simple case.
HttpTestingController	Test double that expects a URL and flushes a response.
canActivate	May this navigation start? Use it for auth UX, not as the only security.
loadComponent	Lazy standalone page. The chunk downloads on navigation.
bypassSecurityTrustHtml	Turns off Angular's escaping. Treat it as a defect unless you can explain the sanitizer.
NgRx	A store and actions. Useful when many features share state. Too much for one form.
SSR	Render HTML on the server for the first paint. I mention it if the job is public and SEO matters. I do not pretend I have run hydration if I have not.

A minute you can rehearse

I build Angular features as standalone pages. The page reads signals, the service does HTTP, and the template uses @if and @for with a track id. Forms that I add now use Signal Forms. Forms already in the app stay on FormGroup until we touch them. Lists that depend on typing use debounce and switchMap. Auth is a guard for the route, an interceptor for the token, and a 401 from the API as the real decision. I can work in an older NgModule app without insisting we rewrite it first.

If you have not used a feature

Say which version your current app is on. Then say the current API in one sentence and stop. A two-year interviewer trusts 'our app is Angular 17 with Reactive Forms, and I know Signal Forms are the API I would use in a new app' more than a recited Angular 22 tour you have never typed.